

CURSO DE SQL Y BASES DE DATOS RELACIONALES

SQL

Inserción, Consulta y Administración de información en bases de
datos relacionales basadas en SQL

Material de Referencia

miyoshikamium@gmail.mx

Versión 6.0 — 8 de julio de 2026

Resumen

Este documento constituye el material teórico de referencia para el Módulo 1 del *Curso de SQL y Bases de Datos Relacionales*. Se abordan los fundamentos históricos y conceptuales que dieron origen a los sistemas de gestión de bases de datos (SGBD) relacionales, la evolución del lenguaje SQL desde sus orígenes en los laboratorios de IBM hasta los estándares modernos, y los principios del modelo relacional propuesto por Edgar F. Codd.

El objetivo es proveer al estudiante una base conceptual sólida que permita contextualizar cada construcción sintáctica de SQL dentro de un marco teórico coherente, facilitando un aprendizaje significativo y duradero. El curso progresa desde la consulta básica de datos, pasando por la modificación y escritura, hasta la administración de esquemas y la integridad referencial.

Palabras clave: base de datos, modelo relacional, SQL, SGBD, álgebra relacional, normalización, ACID, DML, DDL.

Índice

1. ¿Qué es una Base de Datos?	3
1.1. Analogía con el mundo real	3
2. Consulta de Datos: SELECT	4
2.1. Sintaxis completa y cláusulas	4
2.2. Tabla de ejemplo	5
2.3. Proyección de columnas: SELECT * y columnas específicas	5
2.4. Operadores aritméticos	6
2.5. Operadores de comparación	7

3. Modificadores: Filtrado, Ordenamiento y Agrupación	8
3.1. DISTINCT	8
3.2. WHERE	8
3.3. ORDER BY	9
3.4. LIKE	9
3.5. AND, OR, NOT	10
3.6. IN y BETWEEN	11
3.7. GROUP BY y HAVING	11
3.8. CASE	13
3.9. COALESCE e IFNULL	13
3.10. Combinación de tablas: JOIN	13
3.10.1. INNER JOIN	14
3.10.2. LEFT JOIN	14
3.10.3. RIGHT JOIN	15
3.10.4. UNION	15

1. ¿Qué es una Base de Datos?

Definición: Base de Datos

Una **base de datos** es una colección organizada, persistente y compartible de datos relacionados entre sí, diseñada para minimizar la redundancia y maximizar la integridad, de modo que múltiples aplicaciones y usuarios puedan acceder a ella de manera concurrente y controlada **date2003introduction**.

Esta definición merece descomponerse en sus atributos esenciales:

- **Organizada:** Los datos siguen una estructura formal (esquema) que define qué tipos de información se almacenan y cómo se relacionan entre sí.
- **Persistente:** Los datos sobreviven a la terminación de los procesos que los crean. A diferencia de las variables en memoria, los datos en una base de datos permanecen aunque el sistema se apague.
- **Compartible:** Múltiples usuarios o aplicaciones pueden acceder a los mismos datos simultáneamente, de forma controlada y sin interferencias.
- **Mínima redundancia:** El mismo dato no se repite innecesariamente en múltiples lugares, lo que reduce el riesgo de inconsistencias (un dato cambia en un lugar pero no en otro).

1.1. Analogía con el mundo real

Imaginemos el sistema de una biblioteca universitaria:

Ejemplo práctico

Una biblioteca almacena información sobre **libros, estudiantes y préstamos**. Sin una base de datos, cada ficha de préstamo copiaría el título completo del libro, el autor, el nombre del estudiante, su carrera, etc. Si un estudiante cambia su dirección, habría que actualizar decenas de fichas físicas. Con una base de datos relacional, el estudiante aparece *una sola vez* en la tabla **Estudiantes**. Todos los préstamos simplemente hacen referencia a su identificador. Un cambio de dirección se realiza en un único lugar y se refleja automáticamente en todo el sistema.

2. Consulta de Datos: SELECT

La instrucción **SELECT** es, sin duda, la más utilizada y la más rica en posibilidades de todo el lenguaje SQL. Su propósito es **consultar** datos: extraer filas de una o varias tablas, aplicarles filtros, transformaciones y ordenamientos, y devolver un resultado tabular. A diferencia de **INSERT**, **UPDATE** o **DELETE**, **SELECT** es una operación de **sólo lectura**: nunca modifica los datos almacenados.

2.1. Sintaxis completa y cláusulas

La sintaxis formal de **SELECT** en MySQL/SQL estándar es la siguiente. Las partes entre corchetes [] son opcionales; las palabras en mayúsculas son palabras reservadas de SQL:

```

1 SELECT  [ALL | DISTINCT | DISTINCTROW]
2 lista_de_columnas | *
3 FROM    nombre_tabla  [[AS] alias_tabla]
4 [WHERE    condicion]
5 [GROUP BY columna [ASC | DESC], ...]
6 [HAVING   condicion_de_grupo]
7 [ORDER BY columna [ASC | DESC], ...]
8 [LIMIT    numero_de_filas [OFFSET desplazamiento]];
```

Listing 1: Sintaxis completa de **SELECT**.

Cada cláusula tiene un rol preciso dentro de la consulta. La table 1 resume su función y el orden en que el SGBD las *evalúa* internamente (que no siempre coincide con el orden en que se escriben):

Cláusula	Orden eval.	Función
FROM	1	Determina la(s) tabla(s) fuente de datos.
WHERE	2	Filtra filas <i>antes</i> de agrupar.
GROUP BY	3	Agrupar filas con valores iguales en las columnas indicadas.
HAVING	4	Filtra grupos <i>después</i> de agrupar.
SELECT	5	Proyecta (selecciona) las columnas del resultado.
DISTINCT	6	Elimina filas duplicadas del resultado proyectado.
ORDER BY	7	Ordena el resultado final.
LIMIT	8	Recorta el número de filas devueltas.

Tabla 1: Cláusulas de **SELECT** y su orden de evaluación interno.

Nota importante

El orden de **escritura** de las cláusulas es fijo y debe respetarse exactamente como aparece en la sintaxis de arriba. Sin embargo, el SGBD las **evalúa** en un orden diferente (véase table 1). Esto explica, por ejemplo, por qué no se puede

usar un alias definido en **SELECT** dentro de la cláusula **WHERE**: **WHERE** se evalúa *antes* de que exista el alias.

2.2. Tabla de ejemplo

Para ilustrar todos los ejemplos de esta sección usaremos una tabla llamada **empleados** con la siguiente estructura y datos:

```

1 CREATE TABLE empleados (
2   id            INT            PRIMARY KEY AUTO_INCREMENT,
3   nombre        VARCHAR(60)    NOT NULL,
4   departamento  VARCHAR(40)    NOT NULL,
5   cargo         VARCHAR(40)    NOT NULL,
6   salario       DECIMAL(10,2)  NOT NULL,
7   fecha_ingreso DATE           NOT NULL
8 );
9
10 INSERT INTO empleados (nombre, departamento, cargo, salario
11   , fecha_ingreso) VALUES
12 ('Ana Garc a',      'Ventas',      'Gerente',      55000.00,
13   '2018-03-15'),
14 ('Luis Herrera',    'Ventas',      'Analista',     32000.00, '
15   2020-07-01'),
16 ('Sof a Ram rez',   'TI',          'Desarrolladora',
17   48000.00, '2019-11-20'),
18 ('Carlos Mendoza',  'TI',          'DBA',          51000.00, '
19   2017-05-10'),
20 ('Elena Torres',    'RRHH',        'Coordinadora', 38000.00,
   '2021-02-28'),
   ('Marco Villanueva', 'Ventas',      'Analista',     31000.00, '
   2022-09-05'),
   ('Patricia Luna',    'TI',          'Desarrolladora',
   47000.00, '2020-04-14'),
   ('Roberto Salas',    'RRHH',        'Gerente',       52000.00, '
   2016-08-22'),
   ('Diana Castillo',  'Finanzas',    'Analista',     39000.00, '
   2023-01-10'),
   ('Javier Moreno',   'Finanzas',    'Gerente',       58000.00, '
   2015-06-30');

```

Listing 2: Creación e inicialización de la tabla **empleados**.

2.3. Proyección de columnas: **SELECT *** y columnas específicas

La **proyección** es la operación de elegir qué columnas aparecerán en el resultado. El asterisco ***** es un comodín que significa “*todas las columnas*”.

```

1 -- Devuelve todas las columnas de todos los empleados

```

```

2 SELECT *
3 FROM empleados;

```

Listing 3: Seleccionar todas las columnas.

```

1 -- S lo nombre, departamento y salario
2 SELECT nombre, departamento, salario
3 FROM empleados;

```

Listing 4: Seleccionar columnas específicas.

Nota importante

En producción se recomienda **evitar** `SELECT *` porque: (1) transfiere columnas innecesarias por la red, (2) si alguien agrega una columna sensible a la tabla, la consulta la expondrá automáticamente, (3) dificulta la legibilidad del código. Úsalo sólo para exploración interactiva.

También es posible incluir **expresiones** en la lista de columnas: literales, operaciones aritméticas o llamadas a funciones:

```

1 SELECT
2 nombre,
3 salario,
4 salario * 12           AS salario_anual,
5 salario * 0.10         AS bono_estimado,
6 'Activo'               AS estado    -- literal de texto
7 FROM empleados;

```

Listing 5: Columnas calculadas y literales en SELECT.

2.4. Operadores aritméticos

Dentro de `SELECT` (y también en `WHERE`) se pueden usar operadores aritméticos estándar sobre columnas numéricas:

Op.	Nombre	Ejemplo
+	Suma	<code>salario + 5000</code>
-	Resta	<code>salario - descuento</code>
*	Multiplicación	<code>salario * 1.15</code>
/	División	<code>salario / 14</code>
%	Módulo (resto)	<code>id % 2</code> (par/impar)

Tabla 2: Operadores aritméticos en SQL.

```

1 SELECT
2 nombre,
3 salario,

```

```

4 salario * 1.15          AS salario_con_aumento,
5 (salario / 30)          AS salario_diario,
6 salario - 5000          AS salario_base
7 FROM empleados
8 WHERE departamento = 'TI';

```

Listing 6: Uso de operadores aritméticos.

2.5. Operadores de comparación

Los operadores de comparación se usan principalmente en la cláusula WHERE para filtrar filas. Devuelven un valor booleano (TRUE o FALSE):

Operador	Significado	Ejemplo
=	Igual a	departamento = 'Ventas'
<> o !=	Distinto de	cargo <>'Gerente'
>	Mayor que	salario >40000
<	Menor que	salario <35000
>=	Mayor o igual que	salario >= 50000
<=	Menor o igual que	salario <= 48000

Tabla 3: Operadores de comparación en SQL.

```

1  -- Empleados con salario mayor a 45,000
2  SELECT nombre, cargo, salario
3  FROM empleados
4  WHERE salario > 45000;
5
6  -- Empleados que NO son del departamento de TI
7  SELECT nombre, departamento
8  FROM empleados
9  WHERE departamento <> 'TI';
10
11 -- Empleados con salario entre 30,000 y 40,000 (inclusive)
12 SELECT nombre, salario
13 FROM empleados
14 WHERE salario >= 30000
15 AND  salario <= 40000;

```

Listing 7: Filtros con operadores de comparación.

3. Modificadores: Filtrado, Ordenamiento y Agrupación

Los **modificadores** son cláusulas y operadores que se combinan con **SELECT** para refinar, filtrar y ordenar los resultados de una consulta. En esta sección cubrimos los modificadores fundamentales que todo programador SQL debe dominar.

3.1. DISTINCT

Por defecto, **SELECT** devuelve **todas** las filas que satisfacen la consulta, incluyendo duplicados. La palabra clave **DISTINCT** instruye al SGBD a eliminar las filas repetidas del resultado, conservando sólo valores únicos.

Definición: DISTINCT

Modificador que se coloca inmediatamente después de **SELECT** y elimina del resultado todas las filas cuyo conjunto de valores en las columnas proyectadas sea idéntico a otra fila ya presente.

```
1  --    Qu    departamentos existen en la empresa?
2  -- Sin DISTINCT aparecerán 10 filas (una por empleado)
3  SELECT DISTINCT departamento
4  FROM    empleados;
```

Listing 8: DISTINCT sobre una columna.

```
1  -- Combinaciones únicas de departamento + cargo
2  SELECT DISTINCT departamento, cargo
3  FROM    empleados
4  ORDER BY departamento, cargo;
```

Listing 9: DISTINCT sobre múltiples columnas.

Nota importante

DISTINCT opera sobre la *combinación completa* de todas las columnas proyectadas, no columna por columna.

3.2. WHERE

La cláusula **WHERE** es el mecanismo de **filtrado por fila** de SQL. Evalúa una condición booleana para cada fila de la tabla fuente; sólo las filas donde la condición resulta **TRUE** pasan al resultado.

Definición: WHERE

Cláusula que restringe las filas devueltas por una consulta a aquellas que satisfacen una expresión booleana dada. Se evalúa *antes* de **GROUP BY**, **HAVING** y **SELECT**.


```
1  -- Empleados del departamento de TI
2  SELECT nombre, cargo, salario
3  FROM    empleados
4  WHERE   departamento = 'TI';
5
6  -- Empleados con salario mayor a 45,000
7  SELECT nombre, salario
8  FROM    empleados
9  WHERE   salario > 45000;
10
11 -- Empleados que ingresaron después del 1 de enero de 2020
12 SELECT nombre, fecha_ingreso
13 FROM    empleados
14 WHERE   fecha_ingreso > '2020-01-01';
```

Listing 10: Filtros básicos con WHERE.

3.3. ORDER BY

ORDER BY define el **criterio de ordenamiento** del resultado final. Sin él, el SGBD no garantiza ningún orden particular entre ejecuciones.

```
1  -- Orden ascendente (A Z, menor mayor): por defecto
2  SELECT nombre, salario
3  FROM    empleados
4  ORDER BY salario ASC;    -- ASC es opcional, es el valor por
                           defecto
5
6  -- Orden descendente (mayor menor)
7  SELECT nombre, salario
8  FROM    empleados
9  ORDER BY salario DESC;
```

Listing 11: Orden ascendente y descendente.

```
1  -- Primero por departamento ( A Z ),
2  -- dentro de cada departamento por salario ( m a y o r menor )
3  SELECT nombre, departamento, salario
4  FROM    empleados
5  ORDER BY departamento ASC,
6  salario    DESC;
```

Listing 12: Ordenamiento compuesto por múltiples columnas.

3.4. LIKE

LIKE permite comparar una columna de texto contra un **patrón**, usando caracteres comodín. Es la herramienta de búsqueda de texto parcial más básica de SQL.

Comodín	Significado	Ejemplo
%	Cero o más caracteres cualesquiera	'Ana%' coincide con Ana, Analista, Anabel
_	Exactamente un carácter cualquiera	'_na' coincide con Ana, Ina, Una

Tabla 4: Comodines del operador LIKE.

```

1  -- Nombres que empiezan con 'A'
2  SELECT nombre FROM empleados
3  WHERE nombre LIKE 'A%';
4
5  -- Nombres que terminan con 'ez'
6  SELECT nombre FROM empleados
7  WHERE nombre LIKE '%ez';
8
9  -- Nombres que contienen 'ar' en cualquier posición
10 SELECT nombre FROM empleados
11 WHERE nombre LIKE '%ar%';
12
13 -- Nombres que NO contienen 'a' (NOT LIKE)
14 SELECT nombre FROM empleados
15 WHERE nombre NOT LIKE '%a%';

```

Listing 13: Patrones con LIKE.

Nota importante

En MySQL, LIKE es **insensible a mayúsculas/minúsculas** por defecto cuando la columna usa un *collation* insensible. Si necesitas distinguir mayúsculas usa LIKE BINARY.

3.5. AND, OR, NOT

Los operadores lógicos permiten **combinar múltiples condiciones** dentro de una cláusula WHERE.

Operador	Resultado TRUE cuando...	Precedencia
NOT	La condición que sigue es FALSE	Mayor (se evalúa primero)
AND	Ambas condiciones son TRUE	Media
OR	Al menos una condición es TRUE	Menor (se evalúa al final)

Tabla 5: Operadores lógicos y su precedencia.

```

1  -- AND: empleados de TI con salario mayor a 47,000
2  SELECT nombre, departamento, salario

```

```
3 FROM    empleados
4 WHERE    departamento = 'TI'
5 AND      salario > 47000;
6
7 -- OR: empleados de Ventas O de Finanzas
8 SELECT  nombre, departamento
9 FROM    empleados
10 WHERE   departamento = 'Ventas'
11 OR      departamento = 'Finanzas';
12
13 -- NOT: empleados que NO son gerentes
14 SELECT  nombre, cargo
15 FROM    empleados
16 WHERE   NOT cargo = 'Gerente';
17 -- equivalente: WHERE cargo <> 'Gerente'
```

Listing 14: Uso de AND, OR y NOT.

3.6. IN y BETWEEN

```
1 -- Empleados en los departamentos Ventas, TI o Finanzas
2 SELECT  nombre, departamento
3 FROM    empleados
4 WHERE   departamento IN ('Ventas', 'TI', 'Finanzas');
5
6 -- Equivalente usando OR (menos legible)
7 WHERE   departamento = 'Ventas'
8 OR      departamento = 'TI'
9 OR      departamento = 'Finanzas';
```

Listing 15: IN para búsquedas en conjuntos.

```
1 -- Empleados con salario entre 40,000 y 50,000 (inclusive)
2 SELECT  nombre, salario
3 FROM    empleados
4 WHERE   salario BETWEEN 40000 AND 50000;
5
6 -- Equivalente usando AND (menos legible)
7 WHERE   salario >= 40000
8 AND     salario <= 50000;
```

Listing 16: BETWEEN para rangos.

3.7. GROUP BY y HAVING

GROUP BY divide las filas en grupos según los valores de una o más columnas. Típicamente se combina con funciones de **agregación** (COUNT, SUM, AVG, MIN, MAX) para calcular resúmenes por grupo.

Definición: GROUP BY

Cláusula que divide las filas del resultado de **WHERE** en grupos, donde cada grupo comparte el mismo valor en las columnas de agrupamiento. Cada grupo produce exactamente **una fila** en el resultado.

```
1  -- N mero de empleados y salario promedio por departamento
2  SELECT departamento,
3  COUNT(*)           AS num_empleados,
4  ROUND(AVG(salario), 2) AS promedio,
5  SUM(salario)       AS masa_salarial,
6  MIN(salario)       AS minimo,
7  MAX(salario)       AS maximo
8  FROM empleados
9  GROUP BY departamento
10 ORDER BY masa_salarial DESC;
```

Listing 17: GROUP BY con funciones de agregación.

HAVING filtra **grupos** (no filas individuales). Es el equivalente de **WHERE** pero aplicado después de que **GROUP BY** ha formado los grupos.

Definición: HAVING

Cláusula que filtra los grupos producidos por **GROUP BY**, conservando sólo aquellos cuya condición (generalmente sobre una función de agregación) es **TRUE**.

```
1  -- Departamentos donde el salario promedio supera 45,000
2  SELECT departamento,
3  ROUND(AVG(salario), 2) AS promedio
4  FROM empleados
5  GROUP BY departamento
6  HAVING AVG(salario) > 45000;
7
8  -- Departamentos con m s de 2 empleados
9  SELECT departamento,
10 COUNT(*) AS num_empleados
11 FROM empleados
12 GROUP BY departamento
13 HAVING COUNT(*) > 2;
```

Listing 18: HAVING: filtrar grupos por condición agregada.

Nota importante

La diferencia entre **WHERE** y **HAVING**: **WHERE** descarta filas *antes* de agrupar (actúa sobre datos individuales); **HAVING** descarta grupos *después* de agrupar (actúa sobre resultados de agregación).

3.8. CASE

CASE es la expresión de **lógica condicional** de SQL: permite evaluar condiciones dentro de una consulta y devolver valores diferentes según cuál se cumpla.

```
1  -- Clasifica el salario en bandas
2  SELECT nombre,
3  salario,
4  CASE
5  WHEN salario >= 55000 THEN 'Banda A (Senior)'
6  WHEN salario >= 45000 THEN 'Banda B (Mid)'
7  WHEN salario >= 35000 THEN 'Banda C (Junior)'
8  ELSE 'Banda D (Entrada)'
9  END AS banda_salarial
10 FROM empleados
11 ORDER BY salario DESC;
```

Listing 19: CASE buscado (evalúa condiciones booleanas).

3.9. COALESCE e IFNULL

Ambas funciones permiten sustituir un valor NULL por un valor alternativo.

```
1  -- Devuelve el primer contacto disponible entre tres
   opciones
2  SELECT nombre,
3  COALESCE(telefono_celular,
4  telefono_fijo,
5  email,
6  'Sin contacto') AS contacto_preferido
7  FROM empleados;
```

Listing 20: COALESCE: devuelve el primer valor no NULL.

Nota importante

IFNULL(a, b) es equivalente a COALESCE(a, b), pero COALESCE acepta cualquier número de argumentos y es parte del estándar SQL. Prefiere COALESCE para mayor portabilidad.

3.10. Combinación de tablas: JOIN

El operador JOIN combina filas de dos o más tablas basándose en una condición de asociación (típicamente igualdad de clave foránea y clave primaria).

3.10.1. INNER JOIN

Definición: INNER JOIN

Devuelve solo las filas donde existe una coincidencia exacta en ambas tablas. Si una fila de la tabla izquierda no tiene par en la tabla derecha, se descarta completamente.

Sintaxis general:

```
1 SELECT columnas
2 FROM tabla_izquierda
3 INNER JOIN tabla_derecha
4 ON tabla_izquierda.clave = tabla_derecha.clave;
```

Listing 21: Sintaxis de INNER JOIN

Ejemplo práctico

Listar clientes que han realizado pedidos:

```
1 SELECT c.nombre ,
2        p.pedido_id ,
3        p.monto
4 FROM clientes AS c
5 INNER JOIN pedidos AS p
6 ON c.cliente_id = p.cliente_id;
```

Listing 22: INNER JOIN: clientes con pedidos

Solo aparecen clientes que han realizado al menos un pedido.

Nota importante

La palabra clave **INNER** es opcional; **JOIN** a secas equivale a **INNER JOIN** en todos los motores SQL principales.

3.10.2. LEFT JOIN

Definición: LEFT JOIN

Devuelve **todas las filas de la tabla izquierda** más las coincidencias de la tabla derecha. Cuando no existe par, las columnas de la tabla derecha se rellenan con **NULL**.

```
1 SELECT columnas
2 FROM tabla_izquierda
3 LEFT JOIN tabla_derecha
4 ON tabla_izquierda.clave = tabla_derecha.clave;
```

Listing 23: Sintaxis de LEFT JOIN

Ejemplo práctico

Listar todos los clientes y, si tienen pedidos, mostrar su monto:

```
1 SELECT c.nombre ,  
2 p.pedido_id ,  
3 p.monto  
4 FROM clientes AS c  
5 LEFT JOIN pedidos AS p  
6 ON c.cliente_id = p.cliente_id;
```

Listing 24: LEFT JOIN: todos los clientes con sus pedidos

Los clientes sin pedidos aparecen con NULL en las columnas de pedidos.

Caso de uso habitual — detectar registros sin par:

```
1 SELECT c.nombre  
2 FROM clientes AS c  
3 LEFT JOIN pedidos AS p  
4 ON c.cliente_id = p.cliente_id  
5 WHERE p.pedido_id IS NULL;
```

Listing 25: Clientes que nunca han realizado un pedido

3.10.3. RIGHT JOIN

Definición: RIGHT JOIN

Devuelve **todas las filas de la tabla derecha** más las coincidencias de la tabla izquierda. Es el simétrico de LEFT JOIN.

Nota importante

En la práctica, RIGHT JOIN se usa con menos frecuencia que LEFT JOIN. Cualquier consulta con RIGHT JOIN puede reescribirse como un LEFT JOIN intercambiando el orden de las tablas:

$$A \text{ RIGHT JOIN } B \equiv B \text{ LEFT JOIN } A$$

3.10.4. UNION

Definición: UNION

Combina verticalmente los resultados de dos o más sentencias SELECT. UNION elimina filas duplicadas; UNION ALL las conserva.

```
1 SELECT columna1, columna2, ...  
2 FROM tabla_a  
3 UNION [ALL]  
4 SELECT columna1, columna2, ...  
5 FROM tabla_b;
```

Listing 26: Sintaxis de UNION / UNION ALL

Ejemplo práctico

Obtener una lista unificada de correos de dos tablas distintas:

```
1 SELECT nombre, correo, 'cliente' AS origen
2 FROM clientes
3 UNION
4 SELECT nombre, correo, 'proveedor' AS origen
5 FROM proveedores
6 ORDER BY nombre;
```

Listing 27: UNION de correos de clientes y proveedores

Resumen General

Este material ha cubierto los aspectos fundamentales del lenguaje SQL, organizados en torno a tres ejes principales:

1. **Fundamentos teóricos** (secciones 1–4): Historia del problema de los datos, el modelo relacional de Codd, el nacimiento de SQL y la definición conceptual de una base de datos relacional.
2. **Administración de esquemas** (secciones 5–6): Creación y gestión de bases de datos y tablas mediante DDL, incluyendo tipos de datos, restricciones y modificación de estructuras.
3. **Operaciones sobre datos** (secciones 7–9): Consulta (SELECT), escritura (INSERT, UPDATE, DELETE) y manejo de relaciones entre tablas (JOIN, integridad referencial).

El dominio de estos conceptos proporciona la base necesaria para: diseñar esquemas normalizados, escribir consultas eficientes, mantener la integridad de los datos y administrar bases de datos relacionales de forma profesional.